CSC3050 Project 3 – RISC-V Simulator

Semester: Spring 2024

Date: 2025.4.7

Student Information:

金家驹 Jiaju Jin 123090236

蔡礼珂 Like Cai 123090006

目录

Part 0: Introduction to our submission 2

Part 1: Modification of a RISC-V 64-bit simulator 3

Part 1.1: modify RV64 to RV32 3

Part 1.2: realization of fused instructions 3

In MainCPU.cpp: 4

In Simulator.h: 4

In Simulator.cpp: 4

Part 1.3: Disable data forwarding 6

Part 2: Code Schedule 9

Part 2.1 The analysis of the original code 9

Part 2.2 Apply Code rescheduling 9

Part 2.3 Apply Code rescheduling and FMADD.I 10

Part 2.4 Summary of Improvements 11

Part 0: Introduction to our submission

Before the formal report, we would like to introduce our submitted files so that the teaching group can grade our project better.

The submitted files are organized as required by the teaching group. **The src folder and the files in it are uploaded as required, and part2_p2.s and part2_p3.s are also included. Moreover, we also uploaded the latest version of all the results without data forwarding for your reference.** We have successfully implemented all the functions required in part 1 with correct cycle numbers and reduced the cycles of part 2 from 600 to 476 and 464 in p2 and p3 respectively. Hope we can get full marks in this assignment!

Part 1: Modification of a RISC-V 64-bit simulator

Part 1.1: modify RV64 to RV32

To implement a 32-bit simulator, we have to:

1. Use int32_t/uint32_t for all registers
2. Use 32-bit integers for all data operations
3. Leave 64-bit instructions such as LD, SD, ADDW since they are not used in this assignment.
4. Use 32-bit immediate

In fact, just changing all “64” to “32” will satisfy the above requirements, and all basic tests can be successful.

We change all the 64 to 32 in `simulator.cpp` and `simulator.h`, run the `quicksort`, we get corresponding result

```
student@5586892365d5:~/CSC3050_simulator$ ./Simulator riscv-elf/quicksort.riscv
Prev A: 5 3 5 6 7 1 3 5 6 1
Sorted A: 1 1 3 3 5 5 5 6 6 7
Prev B: 100 99 98 97 96 95 94 93 92 91 90 89 88 87 86 85 84 83 82 81 80 79 78 77 76 75 74 73 72 71 70 69 68 67 66 65 64
63 62 61 60 59 58 57 56 55 54 53 52 51 50 49 48 47 46 45 44 43 42 41 40 39 38 37 36 35 34 33 32 31 30 29 28 27 26 25 24
23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1
Sorted B: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40
 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80
 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100
Program exit from an exit() system call
------------ STATISTICS -----------
Number of Instructions: 86136
Number of Cycles: 130446
Avg Cycles per Instrcution: 1.5144
Branch Perdiction Accuacy: 0.4904 (Strategy: Always Not Taken)
Number of Control Hazards: 7366
Number of Data Hazards: 68768
Number of Memory Hazards: 28761
-----------------------------------
```

Figure 1: result of quicksort.cpp

Part 1.2: realization of fused instructions

In this part, we are asked to add 6 more instructions to the simulator.

Table 1: six fused instructions

Inst	Name	funct2	funct3	Description
fmadd.i	Fused Mul-Add	0x0	0x0	rd = rs1 * rs2 + rs3
fmadd.u	Unsigned Fused Mul-Add	0x1	0x0	rd = rs1 * rs2 + rs3
fmsub.i	Fused Mul-Sub	0x2	0x0	rd = rs1 * rs2 - rs3
fmsub.u	Unsigned Fused Mul-Sub	0x3	0x0	rd = rs1 * rs2 - rs3
fnmadd.i	Fused Neg Mul-Add	0x0	0x1	rd = -rs1 * rs2 + rs3
fnmsub.i	Fused Neg Mul-Sub	0x1	0x1	rd = -rs1 * rs2 - rs3

To realize these instructions, we modified following part.

In MainCPU.cpp:

Add the switch to disable data forwarding

```
bool disableDataForwarding = 0;
int main(int argc, char **argv) {
……

    simulator.disableDataForwarding = disableDataForwarding;
……

bool parseParameters(int argc, char **argv) {
  for (int i = 1; i < argc; ++i) {
    if (argv[i][0] == '-') {
      switch (argv[i][1]) {
      case 'x':
        disableDataForwarding = 1; //disable data forwarding.
        break;
```

In Simulator.h:

1. **Add enumeration of instruction**

```
  ...
  SRAW = 53,
  FMADD_I = 54, // Fused Mul-Add (signed)
  FMADD_U = 55, // Fused Mul-Add (unsigned)
  FMSUB_I = 56, // Fused Mul-Sub (signed)
  FMSUB_U = 57, // Fused Mul-Sub (unsigned)
  FNMADD_I = 58, // Fused Neg Mul-Add
  FNMSUB_I = 59, // Fused Neg Mul-Sub
  UNKNOWN = -1,
};
extern const char *INSTNAME[];
```

2. **Add opcode for fused operation.**

```
const int OP_FUSED = 0x0B;  // Custom opcode for fused operations
```

In Simulator.cpp:

1. **Modify array *INSTNAME[]**

```
const char *INSTNAME[]{
    "lui",  "auipc", "jal",   "jalr",  "beq",   "bne",  "blt",  "bge",  "bltu",
      ...
    "lwu",  "slliw", "srliw", "sraiw", "addw",  "subw", "sllw", "srlw", "sraw",
    "fmadd.i", "fmadd.u", "fmsub.i", "fmsub.u",
// fused multiply-add and multiply-subtract
    "fnmadd.i", "fnmsub.i",
// fused negative multiply-add and negative multiply-subtract
};
```

2. **Add Decode logic for OP_R4**

general format of the R4 instruction:

Table 2: format of R4 inst.

rs3	func2	rs2	rs1	func3	rd	opcode
31-27	26-25	24-20	19-15	14-12	11-7	6-0
11	0	10	1	0	100	1011

0x3010203B = 00011 00 00010 00001 000 00100 0001011 for `fmadd.i`

```
case OP_FUSED:{ // layer 1, for opcode
      // get each field of R4 format
      uint32_t funct2 = (inst >> 25) & 0x3;
      uint32_t funct3 = (inst >> 12) & 0x7;
      op1 = this->reg[rs1]; // first operand
      op2 = this->reg[rs2]; // second operand
      op3 = this->reg[rs3]; // third operand (R4 format)
      reg1 = rs1;
      reg2 = rs2;
      reg3 = rs3;
      dest = rd;
        switch (funct3) { // layer 2 in switch case, for funct3
          case 0x0: // fmadd and fmsub series have same funct3
            switch (funct2) { //layer 3 in switch case, for funct2
              case 0x0: // leaf node
                instname = "fmadd.i";  // signed multiply-add
                insttype = FMADD_I;
                break;
```

3. **Add execute logic for R4 inst.**

Take fmadd_I as an example:

```
case FMADD_I:    // rd = rs1 * rs2 + rs3
    writeReg = true;
    out = (op1 * op2) + op3;   // calculation
    this->history.cycleCount += 3;  // 3 more cycles
    break;
```

Do this to all 6 inst.

Then run the bash file, we get result in corresponding folder

Part 1.3: Disable data forwarding

Like Cai encountered some problem in this part, after made modification to execution part, memory part, and write back part, with corresponding stall number 3,2,1. Probably have something to do with main loop, that the program PC change into some value we do not want to see. Due to the wrong PC, the opcode is not the right opcode and get error message "Unsupported opcode 0x7f", while 0x7f is '1111111', which indicates the inst. Increment itself to 32 bits of 1's and stop.

After discussion with Jiaju Jin. We have the following solutions:

1. **Set up -x flag**

The simulator implements data forwarding through a flag
called `disableDataForwarding` that can be controlled via the -x command line option:

In the constructor of simulator

```
Simulator::Simulator(MemoryManager *memory, BranchPredictor *predictor) {
  this->memory = memory;
  this->branchPredictor = predictor;
  this->pc = 0;
  this->disableDataForwarding = false;
```

2. **Key Implementation Details**

The data forwarding logic is implemented in three main places:

1. **Execute Stage (excecute method)**

```
if (!this->disableDataForwarding)
```
// If block handle normal case. After having the if `data forwarding` block

```
1. else{
```

```
2.       if (writeReg && destReg != 0) {
3.         if ((this->dRegNew.rs1 == destReg && this->dRegNew.rs1 != -1)
4.       || (this->dRegNew.rs2 == destReg && this->dRegNew.rs2 != -1)
5.       || (this->dRegNew.rs3 == destReg && this->dRegNew.rs3 != -1)) {
6.           // this->fReg.stall  = 2;
7.           // this->dReg.stall  = 2;
8.           // this->dReg.bubble = true;
9.           // this->history.dataHazardCount++;
10.          this->hazardExist = true;
11.          if (verbose)
12.            printf("  Data Hazard Detected: Stalling Pipeline for %s\n",
   REGNAME[destReg]);
13.        }
14.      }
15.    }
16.
```

To deal with stall addition flag – `hazardExist`

2. Memory Access Stage (memoryAccess method)

```
1. else{
2.       if ((this->dRegNew.rs1 == destReg && this->dRegNew.rs1 != -1)
3.       || (this->dRegNew.rs2 == destReg && this->dRegNew.rs2 != -1)
4.       || (this->dRegNew.rs3 == destReg && this->dRegNew.rs3 != -1)) {
5.         // if (!dReg.stall && !fReg.stall){
6.         // this->fReg.stall  = 1;
7.         // this->dReg.stall  = 1;
8.         // this->dReg.bubble = true;
9.         // this->history.dataHazardCount++;
10.        this->hazardExist = true;
11.        if (verbose)
12.          printf("  Data Hazard Detected: Stalling Pipeline for %s\n",
   REGNAME[destReg]);
13.        // }
14.      }
15.    }
```

Similar structure, with conditional code that either forwards data or marks hazards when forwarding is disabled.

3. Main Simulation Loop

This is the most important part in this method, when the flag `hazardExist` is 1, which mean have encounter some hazard in `exe` or `mem` part, we deal with `stall` and `dataHazardCount` in main loop. Then move PC back to make sure it won't run into unexpected places.
If the PC goes on and still encounter the hazard, the flag is still true, so The pipeline is stalled for one or more cycles until the dependency is resolved.

This method avoid dealing with each hazard case manually, makes logic clearer.

```
if(this->hazardExist)
      if(!this->fRegNew.bubble && !this->dRegNew.bubble){
        this->fReg.stall = 1;
        this->dReg.stall = 1;
        this->dReg.bubble = true;
        this->history.dataHazardCount++;
        this->pc = this->pc - 4;
      }
```

After modifying these, run the bash file and we get what we expected.

Part 2: Code Schedule

Comparison of the Three Assembly Versions and Optimizations Using Code Scheduling and fmadd.i

The three assembly files represent successive optimization stages of the same loop-based program in RISC-V assembly. The goal is to reduce the total number of cycles in a pipeline **without data forwarding** by applying **instruction reordering (code scheduling)** and leveraging **fmadd.i** fused multiply-add instructions.

Part 2.1 The analysis of the original code

The first version (part2.s) directly executes all instructions in sequential order without considering load-use hazards. It includes patterns like:

```
mul a6, a2, a1
sw a6, 0x12(zero)
li a6, 2048
sw a6, 0x36(zero)
li a6, 10
sw a6, 0x20(zero)

mul a4, a1, a2
add a3, a3, a4
```

```
addi a5, a5, 1
sw a5, 0x20(zero)
lw a5, 0x12(zero)

add x6, x7, x6
add x7, zero, x6

mul a6, x8, a3
add a6, a6, a3
```

These cause **load-use hazards**, where an instruction immediately depends on a loaded value before it is written back. In a pipeline without data forwarding, this results in **pipeline stalls** (bubbles), increasing the cycle count.

Part 2.2 Apply Code rescheduling

In the second version (part2_p2.s), code scheduling is applied:

- The order of independent instructions is rearranged to insert unrelated operations between the **load** and its **dependent use**. For example:

```
lw a5, 0x20(zero)  #insert multiple unrelated instructions after loading a5
lw t0, 0x12(zero)
add a3, a3, a4
addi x8, x6, 0
add x6, x7, x6
```

```
addi a5, a5, 1 #here, the a5 is already loaded
```

- The use of the value from li is delayed, giving time for the load to complete and thus avoiding a stall.

This version reduces the number of stalls by strategically placing instructions that do not depend on the loaded values. The total number of cycles is thus reduced **without altering the logic**.

Part 2.3 Apply Code rescheduling and FMADD.I

In the third version (part2_p3.s), further optimization is achieved using .insn pseudo-instructions that represent **fused multiply-add immediate** operations (fmaddi), which are custom or custom-encoded instructions. For example:

```
.insn 4, 0x68c5868b # 0110 1000 1100 0101 1000 0110 1000 1011
#This is for the replacement of
#fmadd a3 a1 a2 a3 == fmadd.i x13 x11 x12 x13

.insn 4, 0x68d4080b # 0110 1000 1101 0100 0000 1000 0000 1011
#This is for the replacement of
#fmadd a6 x8 a3 a3 == fmadd.i x16 x8 x13 x13
```

with a single instruction that does both operations in **three cycle**, reducing instruction count and **eliminating intermediate dependencies** that would have otherwise introduced stalls.

Furthermore, fmaddi allows combining a multiplication and an addition (like a6 = x8 * a3 + a3) into a single instruction, thereby:

- **Reducing instruction count**
- **Avoiding register reuse hazards**
- **Eliminating load-use stalls** by producing results earlier

---

Part 2.4 Summary of Improvements

Version	Key Technique	Data Hazard Handling	Instruction Count	Pipeline Stalls	Cycle Count
part2.s	None (baseline)	None	High	Many	Highest
part2_p2.s	Code Scheduling	Manual reordering	Same as original	Fewer	Reduced
part2_p3.s	Code Scheduling + FMADDI	Combined instructions	Fewer	Fewest	Lowest

By applying **code scheduling**, we delay the use of loaded values and interleave independent instructions, effectively filling pipeline slots that would otherwise be wasted due to stalls. Then, by using **fmaddi**, we merge dependent operations into single instructions, eliminating some hazards entirely.

This results in a more efficient program, even in a pipeline **without data forwarding**.